

Lesson 2

More About Programming



From input to decisions.

-
- 01 Quick review of Lesson 1
 - 02 Input-Process-Output
 - 03 Comments & the compiler
 - 04 Operators: compare & assign
 - 05 If-then-else
 - 06 AND, OR & NOT
-

Every sketch has two functions.

```
void setup() {  
  Serial.begin(9600);  
}  
void loop() {  
  Serial.println("Hello world");  
  delay(1000);  
}
```

SETUP()

Runs once, after each powerup or reset of IA Kit.

LOOP()

Runs continuously. Most of your code goes here.

Six data types.

Type	Meaning	Example
Integer	Integer in Maths, can be positive, 0 or negative	1, 102
Float	Numbers that can have up to 7 decimal places	1.234, 2.345
Double	Numbers that can have up to 15 decimal places	1.234567890123
Char	Single character	A
String	Multiple characters	ABC, Hello
Boolean	A data type whose value is either true or false	true, false

Syntax, run-time & logic errors.

SYNTAX ERROR

```
int SensorValue = 10
void setup() {
  Serial.begin(9600);
  Serial.print (SenserValue);
}
```

Written mistakes stop the code from compiling.

LOGIC ERROR

```
if (sensorValue > threshold) {
  Print("true"); }
else if (sensorValue < threshold) {
  Print("false"); }
```

Nothing would be performed when sensorValue = threshold.

Every program takes in, works on & gives out.

Step 1

Input

The data that we enter or the computer gets from the environment via sensors.



Step 2

Process

Our codes (orders) for the computer to process.



Step 3

Output

The response from the computer after processing the codes.

Useful functions for input.

EXISTED WITHOUT LIBRARY

Input typed by the user in the serial monitor.

```
Serial.parseInt()  
Serial.parseFloat()  
while (Serial.available() > 0){}
```

WITH INCIPE LIBRARY

Input read from the IA Kit's sensors.

```
incipe.getTemperature()  
incipe.getLightIntensity()  
incipe.getDistance()
```

Reading input, line by line.

FROM THE USER

```
Serial.println("Please enter your height.");
```

Ask the user

```
while(Serial.available()==0){}
```

Wait when there's no input

```
int a = Serial.parseInt();
```

Read the input from user and store it in "a"

FROM A SENSOR

```
incipe.main();
```

Activate the sensors & sensor libraries

```
incipe.onscreen("temp1", incipe.getTemperature());
```

Show the temperature in "temp1"

Notes the compiler ignores.

```
String s = "Hello, "; //declare a string

void setup() {
  Serial.begin(9600); //activate Serial Monitor
  Serial.println("What is your name?");
  while (Serial.available() == 0) {
  }
  String input = Serial.readString();
  Serial.print(s + input);
}

void loop() {
  /*
  put your main code here, to run repeatedly:
  */
}
```

Lines in the program that are used to inform yourself or others about the way the program works. They are ignored by the compiler.

// Single-line comment

/* */ Multi-line comment

Think of it like Google Translate.

YOU WRITE

```
Serial.print("Hi");
```



THE COMPILER

Translates the code you write into something the computer can understand and execute.

Meanwhile, it checks if there's any mistake in the written code.



THE COMPUTER

Executes it.

Six ways to compare two values.

COMPARISON OPERATORS

Used to compare the variable on the left with the value or variable on the right.

Symbol	Meaning
<code>==</code>	Equals to
<code>!=</code>	Not equals to
<code><=</code>	Smaller than or equal to
<code>>=</code>	Larger than or equal to
<code><</code>	Smaller than
<code>></code>	Larger than

One equal sign assigns. Two compare.

=

Assign the value on the right to the variable on the left. e.g a=1 means let a equal to 1.

It evaluates the expression on the right first, then stores it in the variable on the left side.

```
int a = 1;
void setup() {
  Serial.begin(9600);
}
void loop() {
  a = a+1;
  Serial.print(a);
  delay(1000);
}
```

=

Compare the variable on the left to the variable on the right.

if runs code when the condition is true.

```
const int number = 5;
// These constants won't change:

void setup() {
  Serial.begin(9600);
  if (number < 10) {
    Serial.println("The number is smaller than 10.");
    // do this order when the condition is true
  }
}
```

IF STATEMENT

It allows you to do something when the condition is true.

Here $5 < 10$, so the message prints.

else runs when the condition is false.

```
float temperature = 27.98;
void setup() {
  Serial.begin(9600);
  if (temperature > 25) {
    Serial.print("turn on");
  } else {
    Serial.print("turn off");
  }
}
```

WHY ELSE

If the primary condition is false, nothing would execute.

So, we create “else” after the primary conditional statement. When the primary condition is false, the statements inside “else” run.

else if adds more conditions.

```
if (someCondition) {  
    // do stuff if the condition is true  
} else if (anotherCondition) {  
    // do stuff only if the first condition is false  
    // and the second condition is true  
} else {  
    // do stuff only when first two conditions are both false  
}
```

if

Only at the first statement.

else if

As many times as required.

else

Only at the last statement, with no condition.

Read the following code. What will be printed?

READ THE CODE

```
int a=10;  
  
if(a<5){a=a+4;}  
else{a=a-5;}  
Serial.print(a);
```

ANSWER

5

WHY

$a < 5$ is false, because 10 is not smaller than 5. So else runs: $a = 10 - 5 = 5$.

Read the following code. What will be printed?

READ THE CODE

```
int x=10;
int y=20;

if (x>y) {
    Serial.println("Hi");
}
else {
    Serial.println("Bye");
}
```

ANSWER

Bye

WHY

$x > y$ is false, because 10 is not larger than 20. So the else block prints Bye.

Combine conditions with three symbols.

&&

AND

Both statements must be true for the computer to process it as true.

||

OR

When any one of the statements is true, the computer processes it as true.

!

NOT

Turns true into false, and false into true.

These three symbols are important when using if-else statements. If a wrong symbol is used, an undesirable result would be presented.

Same numbers, different symbols.

Condition	Result
if ((10 > 7) (90 < 80)) {}	true
if ((10 > 7) (90 > 80)) {}	true
if ((10 > 7) && (90 < 80)) {}	false
if ((10 > 7) && !(90 < 80)) {}	true
if (!(10 > 7) && (90 < 80)) {}	false

READ IT AS

10 > 7 is true.

90 < 80 is false.

Work out each bracket first, then apply the symbol.

Read the following code. What will be printed?

READ THE CODE

```
int x=5;
int y=7;
int z=2;

if(x<y && y<z){
    Serial.print("A");}
else if(x<y || y<z){
    Serial.print("B");}
else{
    Serial.print("C");}
```

ANSWER

B

WHY

$x < y$ is true but $y < z$ ($7 < 2$) is false, so $\&\&$ is false. $\|$ needs only one true, so else if runs and prints B.

Which x, y & z would make this code true?

READ THE CODE

```
if (!(x ≤ y) && (y > z)) {}
```

Give x, y, z respectively.

ANSWER

3, 2, 1

WHY

!(x ≤ y) means x is larger than y, and y > z must also be true. Any x > y > z works, for example 3, 2, 1.

Three things to take away.

- 1 Every program is input, process and output.
- 2 = assigns a value; == compares two values.
- 3 if, else if & else choose what runs; &&, || & ! combine conditions.

Thank you!

LESSON 3

C++ Fundamentals: Conditional (if-else-if) Statements

From the Module 1 curriculum.